

Лабораторна робота №6

Тема: Моніторинг стабільності, продуктивності та проактивне виправлення помилок (Observability & Error Tracking)

Мета роботи: Опанувати принципи технічної спостережуваності (Observability) програмних проєктів. Навчитися інтегрувати системи відстеження помилок (Error Tracking), налаштовувати сповіщення про критичні інциденти та аналізувати продуктивність застосунку для забезпечення безперебійної роботи MVP.

1. Теоретичні відомості

1.1. Технічна спостережуваність (Observability)

Спостережуваність — це здатність розуміти внутрішній стан системи на основі її зовнішніх вихідних даних. Вона базується на трьох "стовпах" (Three Pillars of Observability):

- Логи (Logs):** Записи про події, що відбулися в системі (наприклад, `2024-02-15 10:00:01 User ID 5 logged in`). Корисні для аудиту та детального ретроспективного аналізу, але їх важко моніторити в реальному часі через великий обсяг (Log Noise).
- Метрики (Metrics):** Кількісні показники системи за певний проміжок часу (наприклад, завантаження CPU 80%, 100 HTTP-запитів на хвилину, час відповіді 200 мс). Дозволяють будувати графіки та налаштовувати алерти.
- Трейси (Traces) / Розподілене трасування:** Дозволяють відстежити шлях одного запиту через усі мікросервіси та компоненти системи від початку до кінця. Допомогають знайти "вузьке місце" (bottleneck), де саме відбувається затримка.

Моніторинг помилок (Error Tracking) доповнює ці стовпи, діючи як "сигналізація". Наприклад, інструменти на кшталт Sentry миттєво повідомляють:

```
ReferenceError: user is undefined at checkout.js:45 .
```

Чому це важливо? Користувачі рідко повідомляють про помилки — вони просто йдуть до конкурентів або залишають погані відгуки. Ви повинні дізнаватися про "білий екран" (White Screen of Death) раніше за своїх клієнтів.

1.2. Ключові метрики інцидентів (MTTD та MTTR)

Ефективність команди в реагуванні на проблеми вимірюється двома основними показниками:

- **MTTD (Mean Time To Detect) — Середній час виявлення:** Час від моменту виникнення помилки до моменту, коли команда про неї дізналася. Моніторинг робить цей час мінімальним.
- **MTTR (Mean Time To Repair/Recover) — Середній час відновлення:** Час від виявлення проблеми до випуску виправлення (фіксу). Докладні звіти про помилки (Stack Trace, Breadcrumbs) розшифровують проблему, що радикально зменшує MTTR.

1.3. Контекстне налагодження (Breadcrumbs)

Коли стається помилка, розробнику недостатньо знати лише "Що зламалося?". Найголовніше питання — "Як це відтворити?".

Breadcrumbs (Хлібні крихти) — це автоматичний запис подій та дій користувача, що передували помилці.

Приклад ланцюжка подій:

1. Користувач перейшов на маршрут `/checkout`.
2. Клікнув на кнопку з класом `.btn-pay`.
3. Відправив HTTP-запит `POST /api/pay`.
4. Сервер повернув помилку `500 Internal Server Error`.
5. **BANG!** Сталася помилка в клієнтському JavaScript через відсутність обробки статусу 500.

1.4. Application Performance Monitoring (APM)

Повільний сайт вбиває конверсію так само ефективно, як і зламаний. APM дозволяє відстежувати швидкодію.

- **Latency (Затримка):** Скільки часу сервер або клієнт обробляє подію.
- **Web Vitals (LCP, FID/INP, CLS):** Метрики від Google, що безпосередньо впливають на користувацький досвід та SEO-рейтинг.
 - *LCP (Largest Contentful Paint):* Час до появи найбільшого елемента контенту.
 - *CLS (Cumulative Layout Shift):* Показник візуальної стабільності (наскільки сторінка "стрибає" під час завантаження).
- **Зв'язок з аналітикою (наприклад, PostHog):** Якщо ви бачите падіння конверсії у воронці на етапі "Завантаження каталогу", а в інструменті APM бачите, що API-запит каталогу триває 5 секунд — ви знайшли технічну причину бізнес-проблеми.

1.5. Алертинг та Тріаж (Alerting & Triage)

Не всі помилки однаково критичні.

- **Alerting (Сповіщення):** Автоматичні повідомлення в месенджери (Telegram/Slack) або на пошту. Головна небезпека — "Alert Fatigue" (втома від сповіщень). Якщо система надсилає сповіщення про кожну дрібницю, розробники починають їх ігнорувати. Алерти мають бути налаштовані лише на сплески помилок або на проблеми критичного шляху (наприклад, не працює оплата).
- **Triage (Тріаж / Сорткування розв'язання помилок):** Процес розподілу пріоритетів помилок.
 - *Unresolved:* Нова, ще не розглянута помилка.
 - *Ignored/Muted:* Відома проблема, виправлення якої не є пріоритетним, або це помилка специфічного старого браузера (наприклад, IE11).
 - *Resolved:* Помилку виправлено. Система моніторингу автоматично перевідкриє (Regression) задачу, якщо помилка виникне знову у новій версії.

1.6. Розділення середовищ (Environments) і Source Maps

Розділення середовищ: Життєвий цикл розробки передбачає наявність щонайменше трьох середовищ:

- *Development (dev):* Локальна розробка. Помилки тут — нормальний процес. Їх не потрібно трекати в загальну статистику.
- *Staging (test):* Середовище для тестування (QA-командою).
- *Production (prod):* Робочий продукт для реальних користувачів. Саме тут помилки найбільш критичні.

Налаштування `environment` у системі моніторингу дозволяє фільтрувати помилки та не змішувати логи розробників з проблемами реальних користувачів.

Source Maps (Карти коду): У Production-середовищі код зазвичай мініфікований та обфускований (зокрема JS та CSS) для зменшення розміру. Якщо станеться помилка, звичайний трекер покаже щось на кшталт `Error in a.b.c at main.min.js:1:234`. Це неможливо розшифрувати. *Source Maps* — це спеціальні файли, які маплять (пов'язують) мініфікований код назад у його оригінальний вихідний код, написаний розробником. Вони критично важливі для того, щоб Sentry відображав у Stack Trace зрозумілі назви файлів та рядки (наприклад, `App.vue:45`).

1.7. Zero-Bug Policy

Це філософія управління якістю, яка стверджує: **не можна писати нові фічі, поки існують відомі критичні баги.**

- **Пріоритезація:** Якщо система моніторингу сигналізує про деградацію ключового функціоналу, уся команда призупиняє розробку нових завдань і кидає сили на відновлення працездатності.
- **Боротьба з технічним боргом:** Накопичення дрібних помилок створює ефект "розбитого вікна" та знижує стабільність платформи в довгостроковій перспективі. Zero-Bug Policy дозволяє тримати продукт завжди в стані готовності до релізу.

2. Завдання на роботу (Step-by-Step)

Крок 1: Інтеграція Sentry SDK

Теорія: Sentry діє як "чорна скринька" літака. SDK (Software Development Kit) інтегрується в код застосунку, перехоплює необроблені винятки (unhandled exceptions) та фоново відправляє їх на сервери Sentry для аналізу.

Завдання:

1. Зареєструйтеся на платформі sentry.io.
2. Створіть новий проєкт. Оберіть вашу платформу (наприклад, **Vue**, **React** або звичайний **JavaScript/TypeScript**).
3. Отримайте **DSN (Data Source Name)** — це унікальний ключ-ідентифікатор вашого проєкту, куди SDK буде відправляти дані.
4. Встановіть залежність (приклад для Vue):

```
npm install @sentry/vue --save
```

5. Ініціалізуйте Sentry у головному файлі (наприклад, `main.js` або `main.ts`):

```
import * as Sentry from "@sentry/vue";

Sentry.init({
  app,
  dsn: "ВАШ_DSN_КЛЮЧ",
  integrations: [
    Sentry.browserTracingIntegration(),
    Sentry.replayIntegration(),
  ],
  // Tracing: Записувати 100% транзакцій для моніторингу продуктивності.
  // У продакшені рекомендується зменшити це значення (наприклад, до 0.1), щоб уникнути
  tracesSampleRate: 1.0,
  // Важливо вказувати середовище, щоб відфільтрувати тестові помилки від реальних
  environment: "development", // Змініть на "production" при релізі
});
```

```
11   "format": "prettier --write \"src/**/*.{js,vue,json,css}\"",
12   "test:unit": "vitest",
13   "test:unit:ui": "vitest --ui",
14   "test:unit:coverage": "vitest --coverage",
15   "test:e2e": "playwright test",
16   "test:e2e:ui": "playwright test --ui",
17   "test": "npm run test:unit && npm run test:e2e"
18 },
19 "dependencies": {
20   "@sentry/browser": "^10.38.0",
21   "@sentry/vue": "^10.38.0",
22   "posthog-js": "^1.336.0",
23   "vue": "^3.4.21"
24 },
25 "devDependencies": {
26   "@eslint/js": "^9.39.2",
27   "@playwright/test": "^1.41.1",
28   "@vitejs/plugin-vue": "^5.0.4",
```

Крок 2: Симуляція та відстеження помилок

Теорія: Щоб переконатися, що система перехоплення працює, необхідно штучно викликати виняток і перевірити його появу в панелі керування.

Завдання:

1. Створіть кнопку "Break the world" у будь-якому компоненті (приклад коду для Vue):

```
<button @click="throwError">Викликати помилку</button>
```

```
function throwError() {
  throw new Error("Sentry Test Error: Something went wrong!");
}
```

2. Натисніть створену кнопку. У консолі браузера з'явиться червоне повідомлення про помилку (може також з'явитись білий екран або UI падіння).
3. Перейдіть у дашборд Sentry -> вкладка **Issues**. Ви маєте побачити нову помилку серед "Unresolved".
4. **Аналіз Stack Trace:** Відкрийте інцидент. Sentry підсвітить точний файл і рядок (наприклад, App.vue:45), де виникла помилка. Зверніть увагу на розділ розширеної інформації (теги, інформація про браузер, ОС, IP-адреса).

```
const errorMessage = `Critical failure in show-urgent-filter action: ${urgentTasksCount > 2 ? `too many urgent tasks (${urgentTasksCount})` : ''}`;
const error = new Error(errorMessage)

Sentry.addBreadcrumb({
  message: 'Urgent filter button clicked (experimental 50/50)',
  category: 'user',
  data: { urgentTasksCount, randomValue: randomValue },
})
Sentry.captureException(error)
console.error('[Urgent Filter Error]', error)
throw error
```

The screenshot displays the Sentry dashboard's 'Feed' section. The left sidebar is titled 'Issues' and includes a 'Feed' sub-tab. Below it are links for 'Errors & Outages', 'Breached Metrics', 'Warnings', 'User Feedback', 'All Views', 'Configure', and 'Alerts'. The main content area shows a table of issues. At the top, there are filters for 'All Projects', 'All Envs', and '7D'. A search bar contains the filters 'is unresolved' and 'issue.category is error or outage'. The table has columns: 'Issue', 'Last Seen', 'Age', 'Trend', '24h', '7d', 'Events', 'Users', and 'Assignee'. Three issues are listed:

Issue	Last Seen	Age	Trend	24h	7d	Events	Users	Assignee
Error Critical failure in show-urgent-filter actio... UNIDONE-5 /	1d ago	1d	New	1	0			
Error Critical failure in show-urgent-filter actio... UNIDONE-4 Unhandled / > 4	2d ago	2d	New	7	4			
TypeError posthog.offFeatureFlags is not a function UNIDONE-3 Unhandled / > 3	2d ago	2d	New	4	3			

At the bottom right, it shows '1-3 of 3' with navigation arrows.

Error

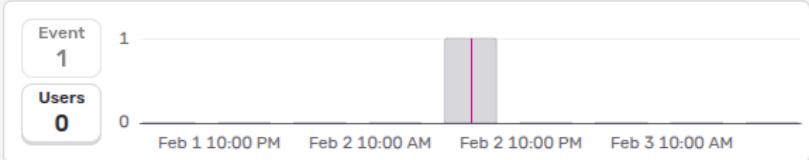
Critical failure in show-urgent-filter action: too many urgent tasks (3)
New /

Events (total) 1
Users (90d) 0

Resolve Archive ...

Priority Assignee Unassigned

All Envs Since First Seen



browser 100% Chrome 144.0.0
url 100% _nidone.vercel.app/
environment 100% production
posthog_id 100% 019c0645-6205-71...
[View all tags and feature flags](#)

Events in this issue < > First Latest Recommended [View More Events](#)

ID: 567e5a13 [a day ago](#) [JSON](#) [Copy as Markdown](#) [Jump to: Highlights](#) [Stack Trace](#) [Breadcrumbs](#) [Trace](#) [Tags](#) [Context](#)

Frontend Chrome 144.0.0 Linux production

Highlights

handled	yes	url	https://unidone.vercel.app/
level	error	Trace: Trace ID	26a11f98d1ae4c97b8fe70a3e6c8f1d7

Stack Trace Most Relevant Full Stack Trace ↑ Newest ...

Error
Critical failure in show-urgent-filter action: too many urgent tasks (3)

mechanism generic handled **true**

/assets/index.DRQJZcQf.js in m at line 47:112516 [In App](#)

```
42  `}}`return`
43
44  {snip} ar e=arguments.length,n=new Array(e>1?e-1:0),r=1;r<e;r++)n[r-1]=arguments[r];return function(s,i)
    {i==void 0&&(i=0);for(var o=[],a=s.split(`
45  {snip} ++u)a[u+10]=l.charCodeAtAt(u)}(i,e),Ya(i,o-8,n.d()),Ya(i,o-4,r),i}var mk=function(t){var
    e,n,r,s,i="";for(e=n=0,r=(t=(t+"").replace(/\r\n/g,`
46  `).replace(/\r/g,`
47  {snip} ve.offFeatureFlags=="function"&&ve.offFeatureFlags(v)})),Lp(==>{g&&g()});const m(==>
    {a.value="urgent";const v=Math.random(),S=l.value.le {snip}
```

/assets/index.DRQJZcQf.js in pi at line 13:38 [In App](#)

/assets/index.DRQJZcQf.js in Yt at line 13:108 [In App](#)

/assets/index.DRQJZcQf.js in HTMLButtonElement.n at line 17:4004 [In App](#)

/assets/index.DRQJZcQf.js in HTMLButtonElement.r at line 33:9862 [In App](#)

Breadcrumbs



Exception - This event error

Feb 2, 6:02:50.551 PM UTC

Error: Critical failure in show-urgent-filter action: too many urgent tasks (3)



User info

Feb 2, 6:02:50.549 PM UTC

Urgent filter button clicked (experimental 50/50)

```
{
  randomValue: 0.9158511062356156,
  urgentTasksCount: 3
}
```



UI Click info

Feb 2, 6:02:50.545 PM UTC

button.px-6.py-2.rounded-lg.font-medium.transition.bg-white.text-gray-700.hover:bg-indigo-50



UI Click info

Feb 2, 6:02:49.454 PM UTC

button.px-6.py-2.rounded-lg.font-medium.transition.bg-white.text-gray-700.hover:bg-indigo-50

Trace Preview

[View Full Trace](#)

	0.00ms	0.00ms
1	Trace — 26a11f98d1ae4c97b8fe70a3e6c8f1d7	
	✓ Error — Critical failure in show-urgent-filter action: too many u	

HTTP Request

GET / [unidone.vercel.app](#)

Headers

Referer <https://vercel.com/>

User-Agent Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36

Tags

[All](#) [Custom](#) [Application](#) [Client](#) [Other](#)

browser Chrome 144.0.0

name Chrome

environment production

handled yes

level error

mechanism generic

os Linux

name Linux

posthog_id 019c0645-6205-7117-9b46-ac40ad8cedc6

transaction /

url <https://unidone.vercel.app/>

Contexts

User

Geography Kyiv, Ukraine (UA)

Operating System

Name Linux

Browser

Name Chrome

Version 144.0.0

Trace Details

Client Sample Rate 0.2

Span ID 9fddfa74bde4bcf2

Status unknown

Trace ID [26a11f98d1ae4c97b8fe70a3e6c8f1d7](#)

Крок 3: Налаштування контексту користувача та тегів

Теорія: Безіменна помилка ("Cannot read property of undefined") дає мало розуміння проблеми. Якщо ж ви знаєте, що помилка виникла у конкретного користувача ("User ID 55, Premium"), ви можете швидше відтворити її в його профілі та навіть проактивно зв'язатися з ним з вибаченнями ще до того, як він напише в підтримку.

Завдання:

1. Додайте ідентифікацію користувача. У місці вашого коду, де відбувається успішна авторизація (наприклад, після логіну), передайте дані в Sentry:

```
// Дані користувача зазвичай беруться зі стейт-менеджера (Pinia/Redux) або JWT токена
Sentry.setUser({
  id: "12345",
  email: "student@example.com",
  segment: "premium_user" // Приклад кастомного тегу для сегментації
});
```

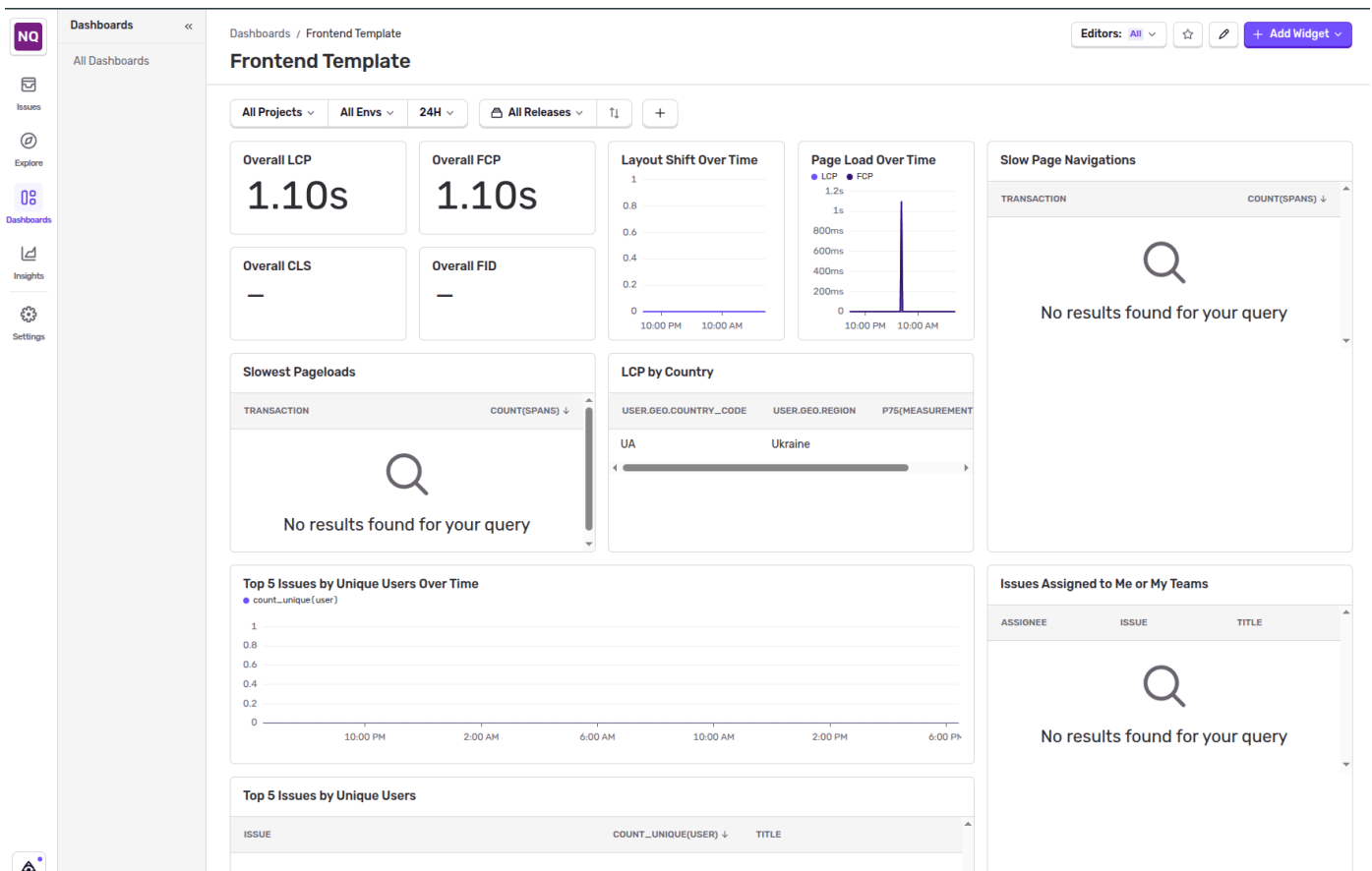
2. Викличте помилку ще раз і переконайтеся, що в деталях помилки в Sentry тепер відображається email та ID користувача.
3. *Очищення контексту:* Не забудьте викликати `Sentry.setUser(null)` при виході користувача з системи (Logout), щоб уникнути змішування сесій.

Крок 4: Моніторинг продуктивності (APM & Tracing)

Теорія: Транзакція (Transaction) в APM — це вимірювання повного циклу певної операції (наприклад, "Завантаження сторінки Завдань" або "Реєстрація"). Вона складається з дрібніших операцій — спанів (Spans), таких як рендеринг конкретного компонента або очікування HTTP-запиту до бекенду.

Завдання:

1. Переконайтеся, що у налаштуваннях `Sentry.init` параметр `tracesSampleRate` увімкнено (наприклад, `1.0` під час тестування).
2. Перейдіть у Sentry у вкладку **Performance**. Тут збирається статистика Web Vitals та загальна інформація про швидкість завантаження маршрутів (Routes).
3. **Глобальні метрики (Web Vitals):**



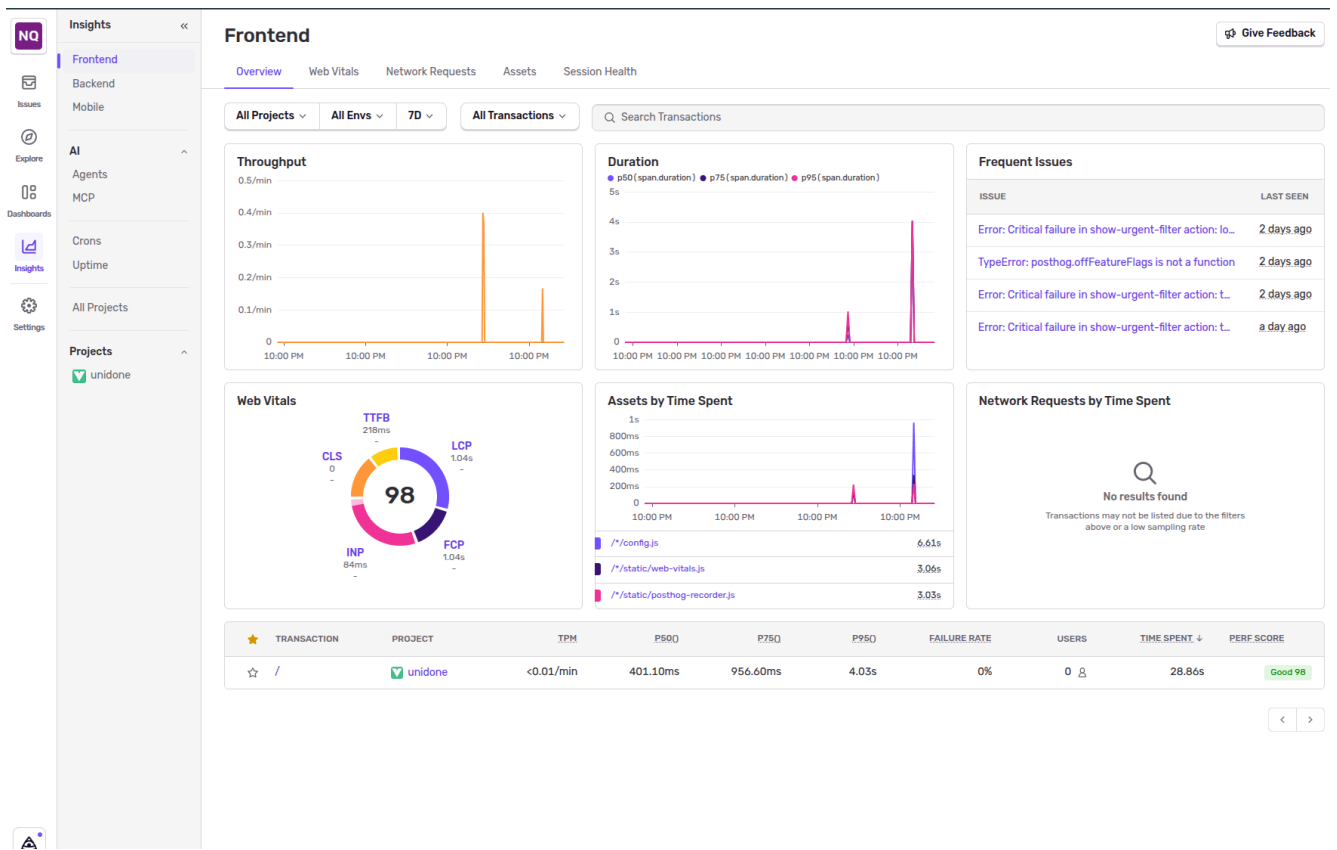
- **LCP (Largest Contentful Paint):** Оцініть час до відображення основного контенту (норма < 2.5с).
- **CLS (Cumulative Layout Shift):** Оцініть стабільність верстки (норма < 0.1).
- **High Latency:** Знайдіть маршрути з найбільшою затримкою.

4. Оберіть найповільнішу транзакцію (наприклад, ваш умовний GET /tasks або завантаження важкого UI елемента).

5. Подивіться "**Waterfall**" діаграму спанів. Визначте, на що витрачається найбільше часу.

Аналіз показників:

- *Throughput (Пропускна здатність):* Кількість транзакцій, що обробляються, за одиницю часу.
- *Duration (P95):* Якщо p95 = 4с, це означає, що 95% користувачів отримують відповідь швидше ніж за 4 секунди, а для 5% користувачів завантаження триває довше (найбільш проблемні кейси).



Крок 5: Налаштування алертингу (Alert Rules)

Теорія: Ніхто не моніторить дашборди безперервно. Автоматизована система сповіщень дозволяє негайно реагувати на аномалії (наприклад, різке зростання кількості 500-х помилок після нового деплою).

Завдання:

1. Перейдіть у меню **Alerts** -> **Create Alert Rule**.
2. Оберіть тип алерту (наприклад, *Issues*).
3. Налаштуйте умову (Condition). Наприклад:
When count of unique errors is more than 5 in 1 minute (Коли кількість унікальних помилок перевищує 5 за хвилину).
4. Налаштуйте дію (Action) — Send notification to Mail (або в Slack/Telegram, якщо у вас налаштована інтеграція).
5. **Перевірка:** Швидко натисніть на створену раніше кнопку генерації помилки 10-15 разів поспіль.
6. Перевірте вашу пошту (або обраний канал комунікації) — туди має надійти лист-сповіщення про перевищення порогу помилок (Alert trigger).

NQ

Issues

Feed

Errors & Outages

Breached Metrics

Warnings

User Feedback

All Views

Configure

Alerts

Settings

Alerts

Create AlertGive Feedback

Alert RulesHistory

My Teams +1All ProjectsAlert Type: AllSearch by name

Alert Rule	Status	Project	Team	Actions
Send a notification for high priority issues Triggered a day ago		undone		

Issues

Feed

Errors & Outages

Breached Metrics

Warnings

User Feedback

All Views

Configure

Alerts

Alerts / Edit Alert Rule

Send a notification for high priority issues

1 Select an environment and project

All Environmentsundone

2 Set conditions

WHEN an event is captured by Sentry and any of the following happens

Sentry marks a new issue as high priority

Sentry marks an existing issue as high priority

Add optional trigger...

IF all of these filters match

Add optional filter...

THEN perform these actions

Send a notification to Suggested Assignees and if none can be found then send a notification to Recently Active M...

Add action...

Send Test Notification

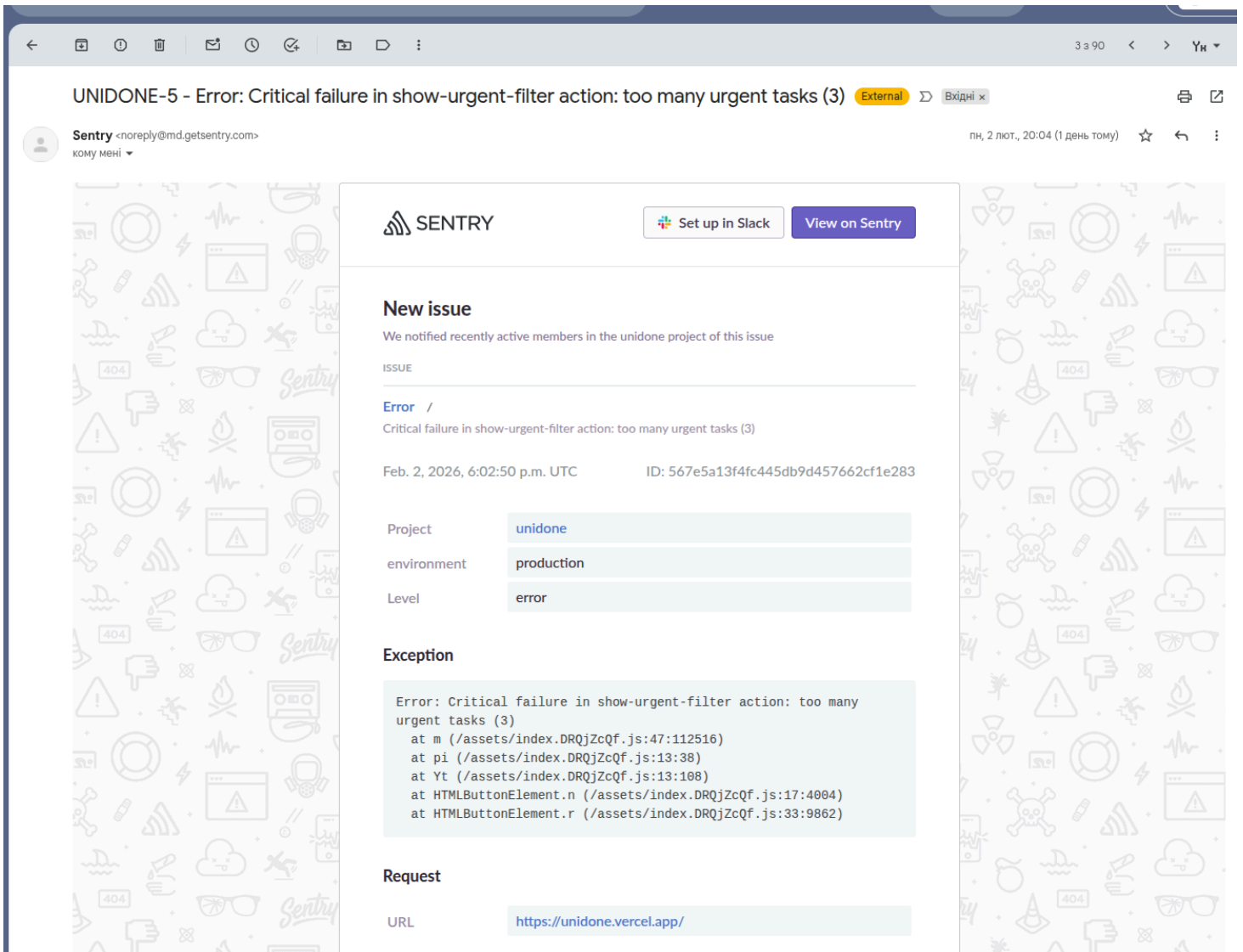
3 Set action interval

Perform the actions above once this often for an issue

30 minutes

4 Preview

3 issues would have triggered this rule in the past 14 days approximately. If you're looking to reduce noise then make sure to read the docs.



3. Вимоги до звіту

1. Скриншот детальної інформації про згенеровану помилку в Sentry. На скриншоті має бути чітко видно **Stack Trace** (точне місце у коді) та **Breadcrumbs** (дії користувача, що передували помилці).
2. Скриншот, що підтверджує успішну передачу контексту користувача (email, ID) до деталей інциденту в Sentry.
3. Скриншот дашборду **Performance** з відображенням загальних метрик затримки (Latency / Web Vitals).
4. Скриншот налаштованого правила алертингу (умови спрацювання) та скриншот отриманого сповіщення (листа на пошті, повідомлення в Telegram тощо).
5. **Аналітичний висновок:** Напишіть короткий висновок з моделюванням ситуації. Як інформація з систем Error Tracking (технічна проблема) може доповнювати дані з Product Analytics (наприклад, PostHog - відтік користувачів на воронці). *Приклад сценарію:*

"Аналітика показує, що 40% користувачів не завершують реєстрацію. Проглянувши Sentry, ми виявили критичну помилку CORS на другому кроці реєстрації, яка є справжньою причиною цього відтоку".

4. Контрольні запитання

1. У чому полягає різниця між традиційним логуванням (logging) і моніторингом помилок (error tracking)?
2. Які три основні "стовпи" (Three Pillars) лежать в основі концепції технічної спостережуваності (Observability)?
3. Що таке Source Maps і чому вони критично важливі для відстеження помилок фронтенду в Production-середовищі?
4. Як використання інструментів на кшталт Sentry та налаштування сповіщень впливають на ключові показники DevOps — MTTD (Mean Time To Detect) та MTTR (Mean Time To Repair)?
5. Чому вважається обов'язковим розділяти середовища розробки (Development, Staging, Production) в конфігурації системи моніторингу?
6. Що таке "Alert Fatigue" (втома від сповіщень) і якими методами їй можна запобігти?